

Google DeepMind

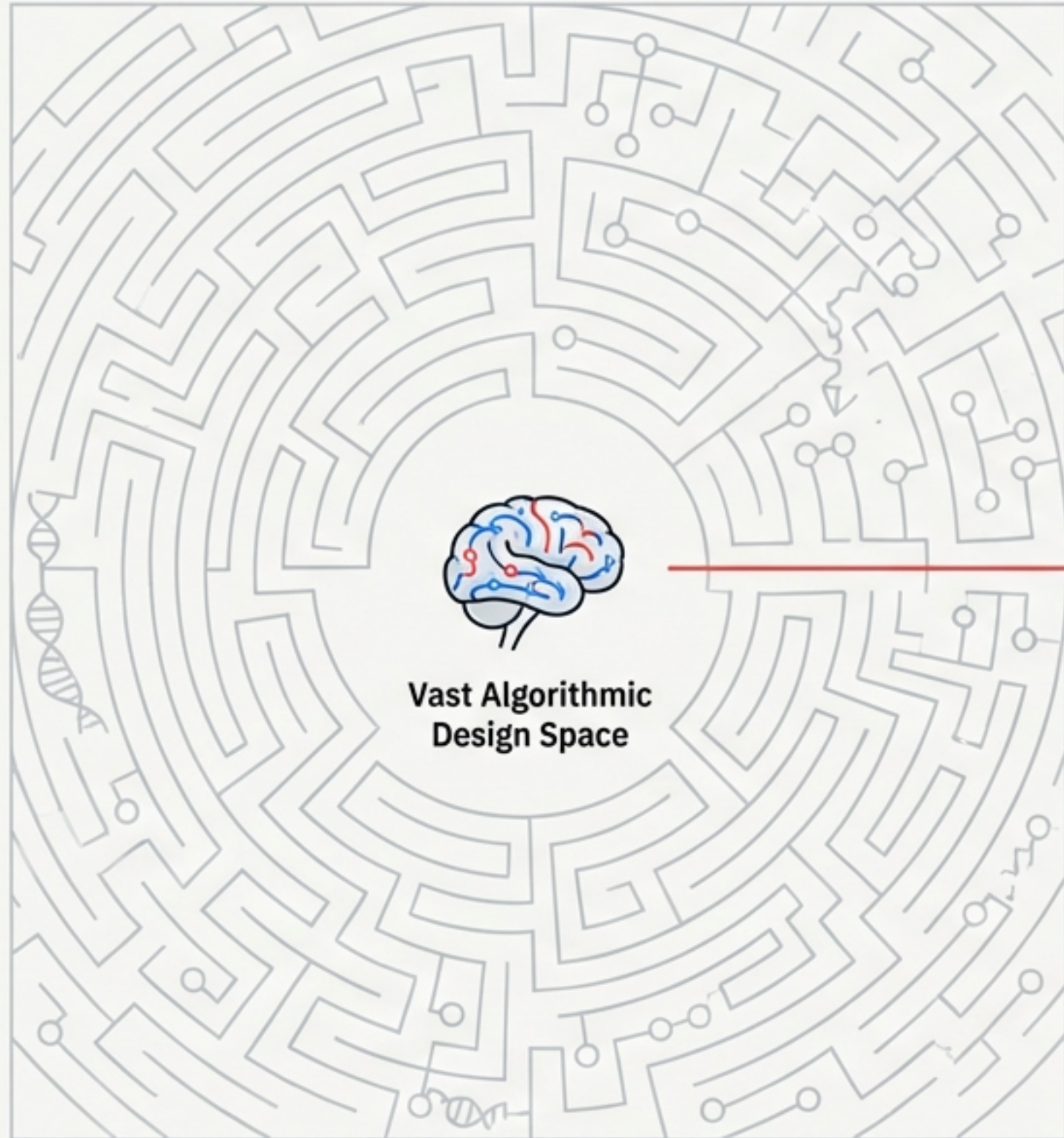


AI สร้าง AI: ก้าวข้ามขีดจำกัดของมนุษย์ในการออกแบบอัลกอริทึม

สรุปงานวิจัย Discovering Multiagent Learning Algorithms with Large Language Models
โดย Google DeepMind (2026)

เมื่อ Large Language Models กลายเป็นวิศวกรผู้วิวัฒนาการโค้ดเพื่อแก้ปัญห Game Theory ที่ซับซ้อนที่สุด

ทางตันของ สัญชาตญาณมนุษย์ ใน Multi-Agent RL



1. Manual Refinement (การลองผิดลองถูก)

นักวิจัยต้องใช้เวลาไปกับการคาดเดาและปรับจูนโครงสร้างด้วยมือเปล่า



2. Tractable Heuristics (กับดักความเรียบง่าย)

มนุษย์มักเลือกใช้สมการคณิตศาสตร์ที่ดูเรียบง่ายและเข้าใจได้ เช่น การใช้ค่าเฉลี่ยเชิงเส้น หรือส่วนลดคงที่



3. Missed Opportunities (ประสิทธิภาพที่ถูกมองข้าม)

โครงสร้างอัลกอริทึมที่ซับซ้อนและมีประสิทธิภาพสูงกว่ากลับไม่ถูกค้นพบ เพราะมักจะขัดกับสามัญสำนึก (Non-intuitive) ของมนุษย์

AlphaEvolve: เปลี่ยน LLM ให้เป็น ผู้สร้างวิวัฒนาการ

ยกระดับจากการจูนพารามิเตอร์ สู่การทำ Semantic Evolution

Parameter
Tuning



Semantic
Evolution



```
def mutate(code):  
    if x > y:  
        code = return z  
        code.x = y  
        return z + + y  
  
    return z
```

```
def mutate(code):  
    code = [];  
    if x > y: return z  
        code.removeforr(code);  
    else x > {{a  
    return z
```

```
def mutate(code):  
    if x > y:  
        code = return  
        return z  
  
    else:  
        x = mitan  
        code = "y +  
        1 - y > +
```

```
return = 3:
```

1 Concept 🧬

มองว่า Source Code
ของอัลกอริทึมคือยีน
(Genome)

2 Action 🧠

LLM ทำหน้าที่เป็น Intelligent
Genetic Operators

✓ Result 🚀

เขียนลอจิกใหม่, สร้าง
Control flows รูปแบบใหม่,
และใส่สมการที่มนุษย์คิดไม่ถึง

วงจรการค้นพบแบบอัตโนมัติ (Automated Discovery Loop)

1. Initialization (ยีนตั้งต้น)

```
fira mutate(code, => Uniform PSRO))
```

เริ่มต้นด้วย Source code มาตรฐานที่มนุษย์เขียนไว้ (เช่น CFR หรือ Uniform PSRO)

```
def mutate(code):  
    if x > y:  
        code = return z  
  
    else:  
        code.x= {  
            rmanform(code).*
```

4. Evolutionary Selection (การสืบทอดเผ่าพันธุ์)

คัดเลือกอัลกอริทึมที่รอดชีวิตและทำคะแนนได้ดีที่สุดกลับเข้าสู่ประชากรเพื่อวนลูปวิวัฒนาการต่อไป



2. LLM-Driven Mutation (การกลายพันธุ์)



(การกลายพันธุ์)

ส่งโค้ดให้ LLM เสนอการปรับแก้โค้ดเชิงความหมาย (Semantic modification) เพื่อเพิ่มประสิทธิภาพ

3. Automated Evaluation (การคัดเลือกรุ่น)



นำโค้ดใหม่ไปทดสอบรันจริงกับเกมจำลอง (Proxy games เช่น Kuhn Poker) เพื่อหาค่า Exploitability

```
return = 3;
```


บทพิสูจน์ใน 2 กระบวนทัศน์หลักของ Game Theory



Regret Minimization (CFR)

- โฟกัสที่การคำนวณและอัปเดตความเสียหาย (Regret) เพื่อหาจุดสมดุลในเกมขนาดเล็ก-กลาง (เช่น ปุ๊กเกอร์)



The Discovery: VAD-CFR
(Volatility-Adaptive Discounted CFR)



Population Based Training (PSRO)

- โฟกัสที่การสร้างกลุ่มประชากรกลยุทธิ์ (Meta-Game) สำหรับเกมขนาดใหญ่ที่มีกิ่งก้านมหาศาล



The Discovery: SHOR-PSRO
(Smoothed Hybrid Optimistic Regret PSRO)

การค้นพบที่ 1: VAD-CFR

ทั้งสมการตายตัว สู่กลไกการเรียนรู้ที่ขัดกับสัญชาตญาณมนุษย์

AlphaEvolve ถูกปล่อยให้ค้นหาวิธีการสะสม Regret และหาค่าเฉลี่ยนโยบาย (Policy) สิ่งที่มีนัยสำคัญคือการทั้งสมการเชิงเส้นแบบเดิม และสร้างอัลกอริทึมใหม่ VAD-CFR ที่มีความเป็นพลวัตสูงมาก แม้กลไกจะดูประหลาดในสายตาตามมนุษย์ แต่กลับทำงานได้ดีกว่า State-of-the-Art ทุกตัว

Human Baseline (DCFR)

$$R_{t+1} = (R_t * \alpha) + (r_t * \beta)$$

Evolved Discovery (VAD-CFR)

```
def vad_cfr(r, R):  
    if r > 0: -----  
        R = max(0, R + r)  
    else:  
        R = min(R + r, R * 0.95) -----  
    return R
```

Dynamic Adaptation

3 กลไกอัจฉริยะเบื้องหลัง VAD-CFR

```
Fira Code ({
  function VAD(){
    var sistenage = bookPran
    prvmom: math(),
    maxlt: #dVemane.setUnawO
    _weunt: { Policy iwemw
  }
})
conz.ciernetProp() {
  var laeysen _icvn = bookPoal
```

Volatility-Adaptive Discounting



เข็มไมล์คอยจับตาดูความผันผวน
หากความผันผวนสูง อัลกอริทึมจะเพิ่ม
อัตราการลืมอดิตที่พึงพาไม่ได้ให้เร็วขึ้น
หากนิ่งแล้วจึงค่อยจดจำ

Asymmetric Boosting



มนุษย์ชอบออกแบบอัปเดตแบบสมมาตร
แต่ AI เลือกให้โบนัพิเศษแบบไม่สมมาตร
ทันทีกับ Action ที่เป็นบวกในปัจจุบัน
เพื่อตักตวงผลประโยชน์โดยไม่ต้องรอ

Hard Warm-Start



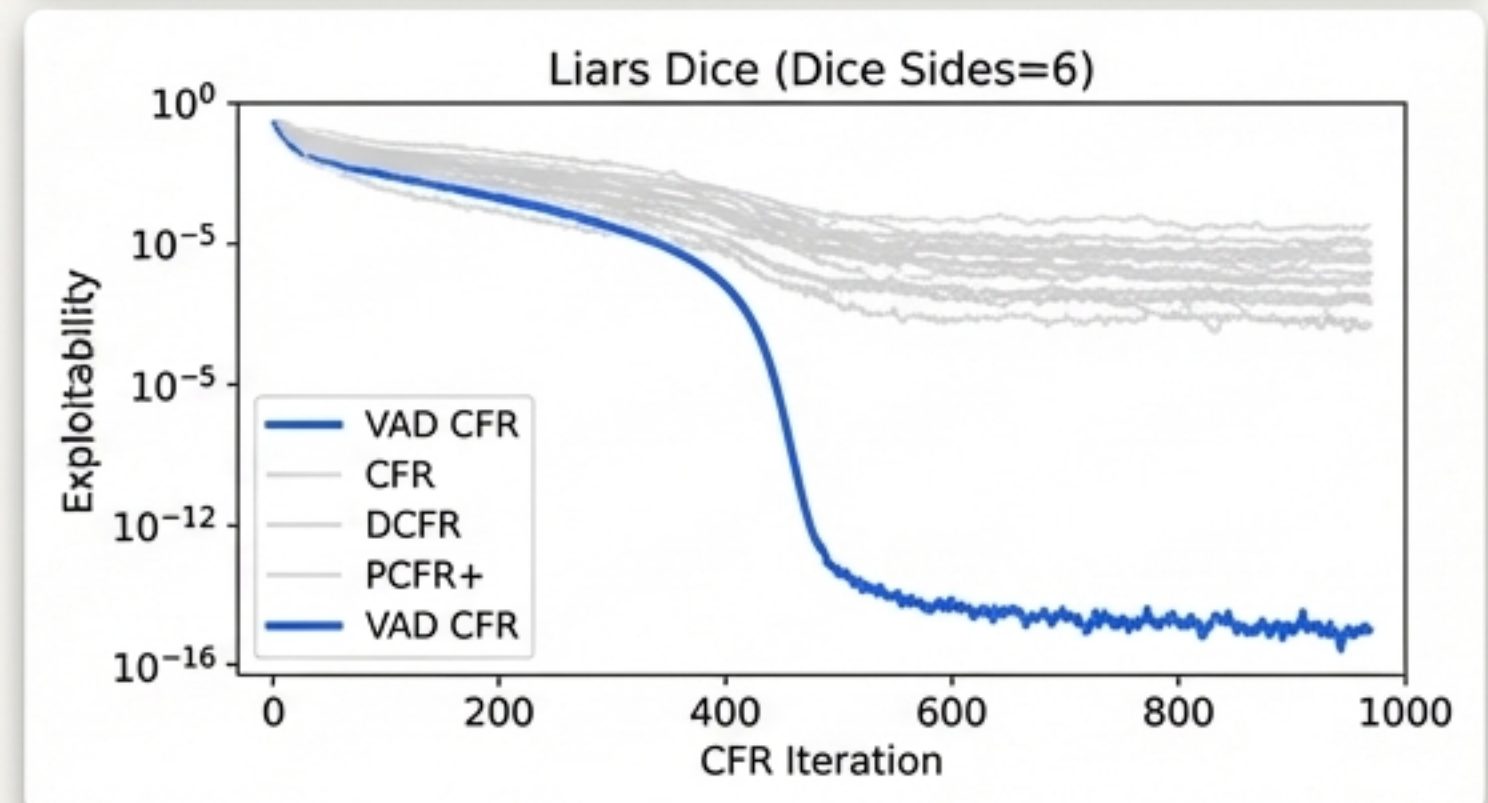
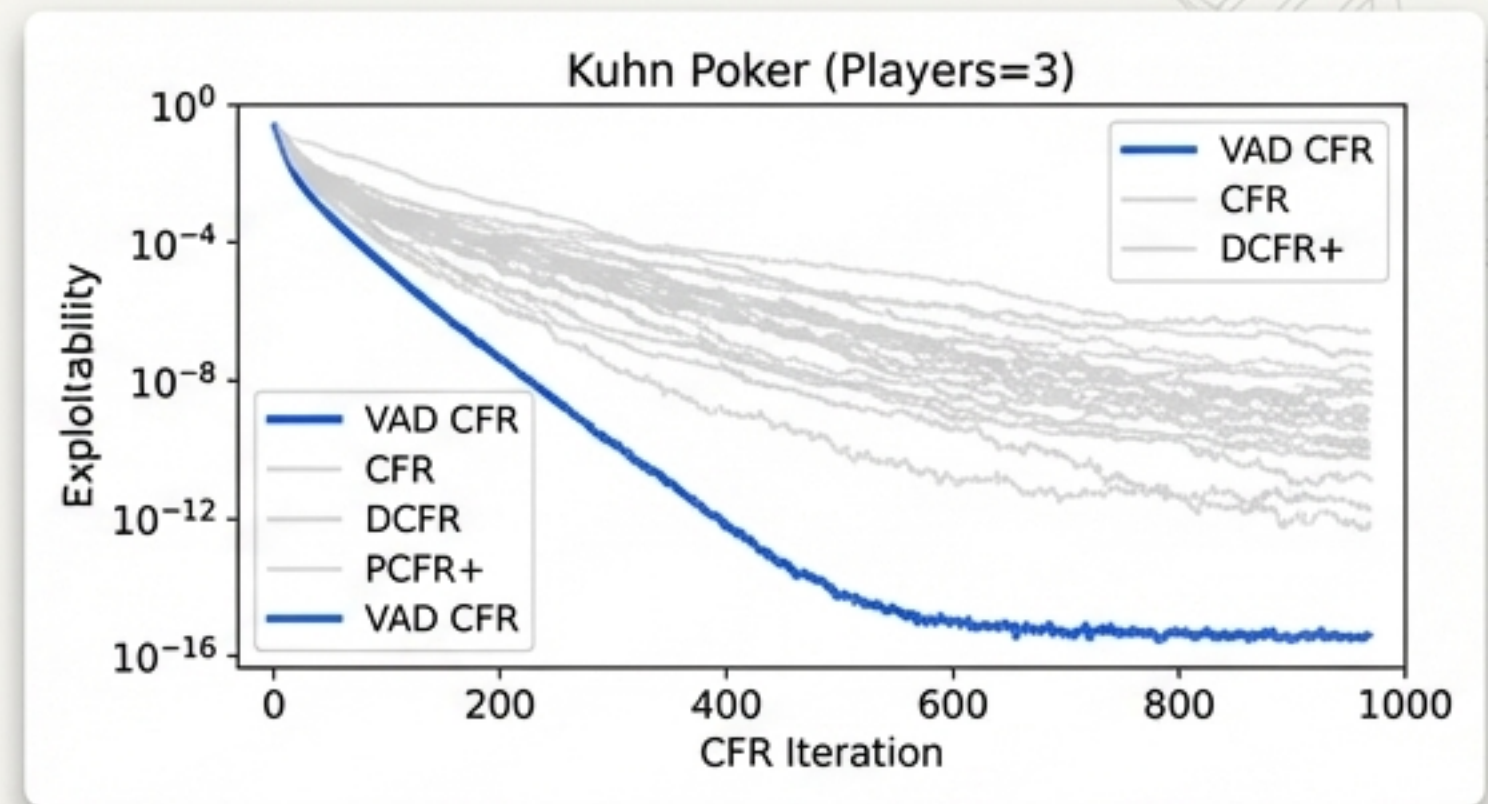
แทนที่จะเริ่มหาค่าเฉลี่ยทันที
AI เลือกที่จะหยุดการจดจำ Policy
ในช่วง 500 Iteration แรก
แล้วจึงเริ่มบันทึกเฉพาะข้อมูลที่มีคุณภาพ
(Magnitude-weighted)

ประสิทธิภาพของ VAD-CFR: ทิ้งห่างทุก Baseline

```
Fira Code {  
  for i in 1000 {  
    let mm = rankRateSur($ths);  
    return {  
      gvr = amravsEtsant([x,""], x.ann));  
      if returns ...  
    }  
  }
```

- **Training Set** (Kuhn/Leduc Poker):
VAD-CFR ทำค่า Exploitability ได้ต่ำกว่า
และ Converge เร็วกว่าอย่างเห็นได้ชัด
- **Test Set** (Generalization - Liar's Dice):
สามารถนำไปประยุกต์ใช้กับเกมที่ใหญ่และ
ซับซ้อนกว่าได้อย่างยอดเยี่ยม ทนทานต่อ
Noise ได้ดีกว่าอัลกอริทึมที่มนุษย์สร้าง

```
Test(s: Players=3)  
{  
  if = wiot {  
    retur an($sesi.zōs>38)  
  }  
  case ontration {  
    VAD-CFR = energvyIwd, em(,n[]))  
  }  
}
```



การค้นพบที่ 2: SHOR-PSRO

ยกระดับจาก Meta-solver แบบตายตัว สู่ Hybrid Engine

ในฝั่ง PSRO ปัญหาเดิมคือการที่นักวิจัยต้องเลือกระหว่าง การสำรวจ (Exploration) หรือ การแสวงหาผลประโยชน์ (Exploitation) แบบตายตัว AlphaEvolve ได้คลายข้อจำกัดนี้โดยค้นพบ SHOR-PSRO ซึ่งเป็นลูกผสมที่ปรับสมดุลตัวเองแบบไดนามิกตลอดการเทรน

Static Meta-solvers



Uniform
(Exploration)



Nash
(Exploitation)

Dynamic Meta-solver (SHOR-PSRO)

Uniform/ORM

Hybrid Engine



Nash/Softmax

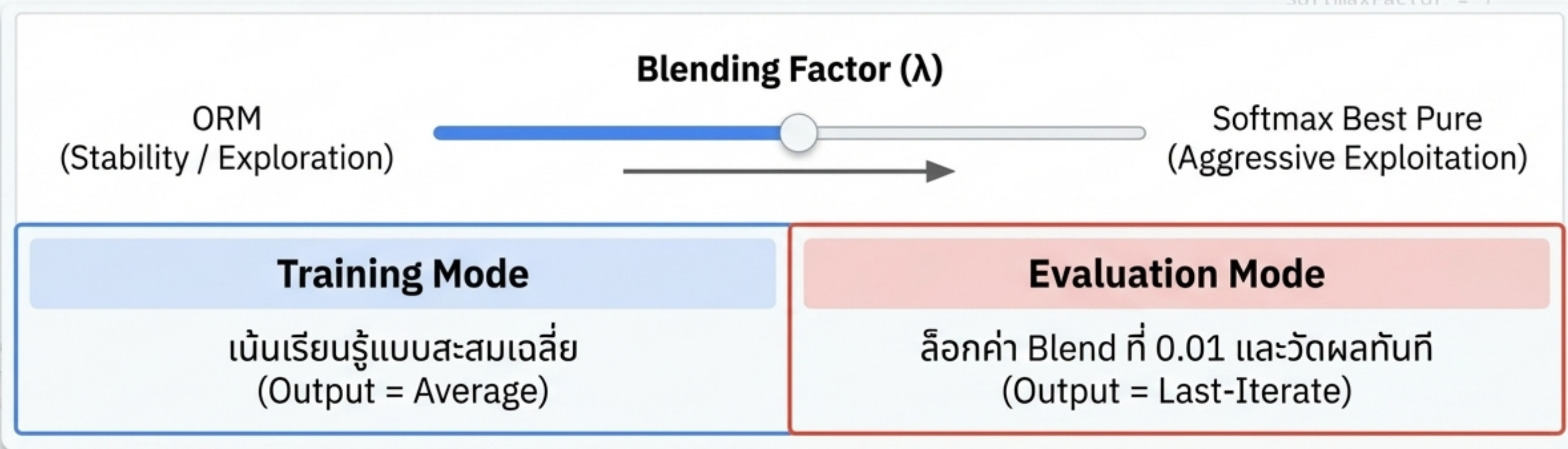
Gearbox



**Adaptive
Solution**

กลไก Dynamic Annealing & Hybrid Blending

```
# Fira Code
# const code = ...
const &z = &codor
const pure = Softmax
```



Hybrid Equation:

ผสมผสานความเสถียรของ ORM เข้ากับความดุเดือดของ Smoothed Best Pure Strategy

Auto-Annealing:

ปรับลดสัดส่วนการผสมอัตโนมัติ เริ่มด้วยการหาความหลากหลาย จบด้วยการขัดเกลาจุดสมดุล

Decoupled Modes:

แยกโหมด Training และ Evaluation ออกจากกันอย่างเด็ดขาด เพื่อลด Noise ในการวัดผล

```
for ([raa = 1] conator; i < blenning; i++):
  Softmax_λ = "Best Softmax object"
  conwiderent(Softmax_Straegy)
```


SHOR-PSRO: เร็วกว่า เสถียรกว่า ทนทานต่อความซับซ้อน

เอาชนะ Meta-solvers แบบดั้งเดิม: เอาชนะ Meta-solvers แบบดั้งเดิมได้อย่างราบคาบ โดยเฉพาะในเกมที่มีกึ่งก้านการตัดสินใจมหาศาล (เช่น 6-sided Liar's Dice)

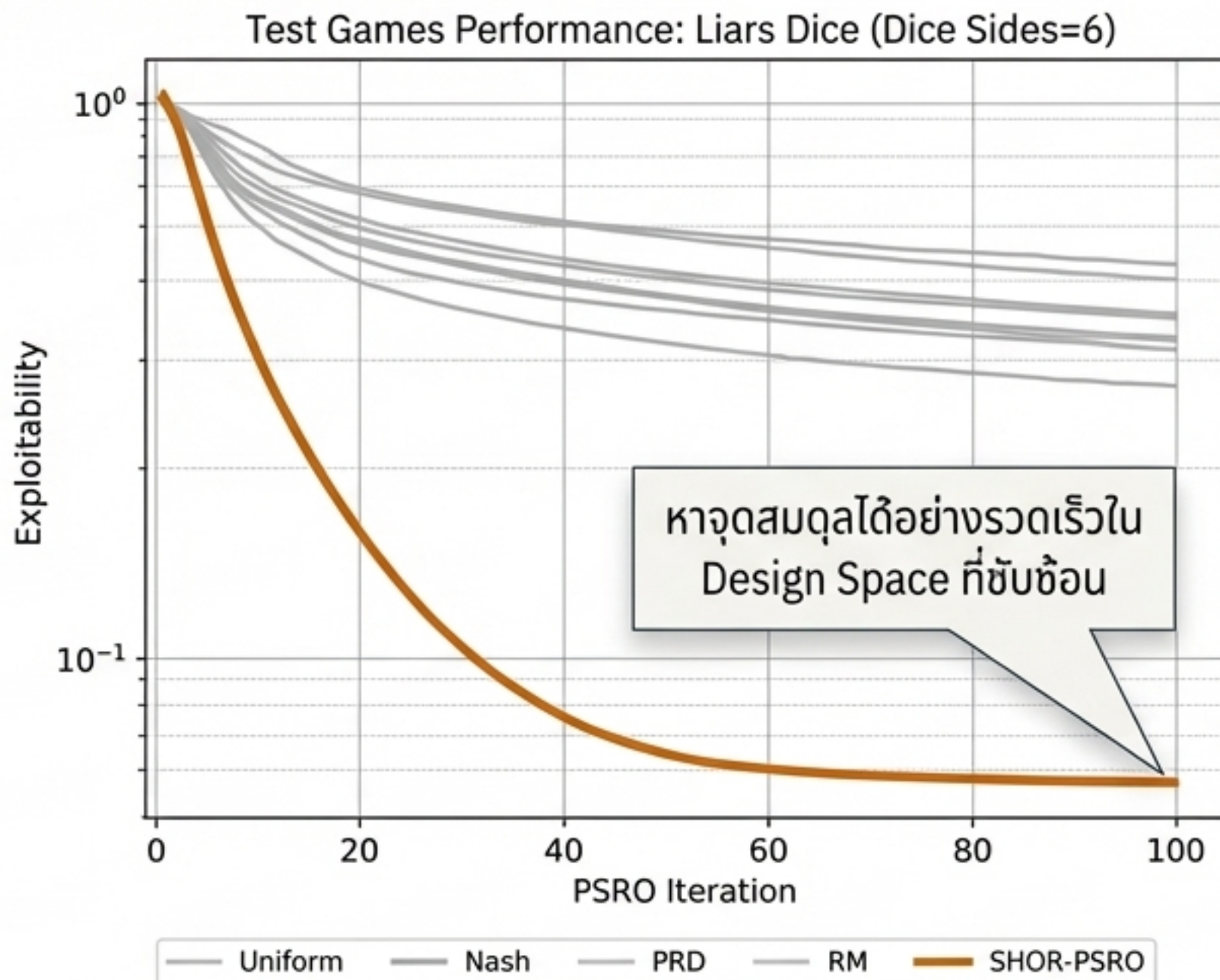
Complex Graph Navigation: ความสามารถในการนำทางของไฮบริดโมเดลทำให้หาจุดสมดุลได้ไวแบบก้าวกระโดด ในขณะที่ Solver แบบเก่าติดหล่มอยู่กับการสำรวจ

```
Fira Code ss <firaarsz8>
```

```
data solver_rada=ms.1  
enetlm = "IR"  
towlnerent_exploitranwe =B,zSkd,l,  
cost:ends_ronat_of {
```

```
# Fira Code ===  
#import stem  
lcnn stem,tn = 1dAtte
```

```
for _solver in item) {  
  Stonarsolvers<->wertRenant("VTe tas 200 s2ders")  
  reaults = nonatesolver(41)
```

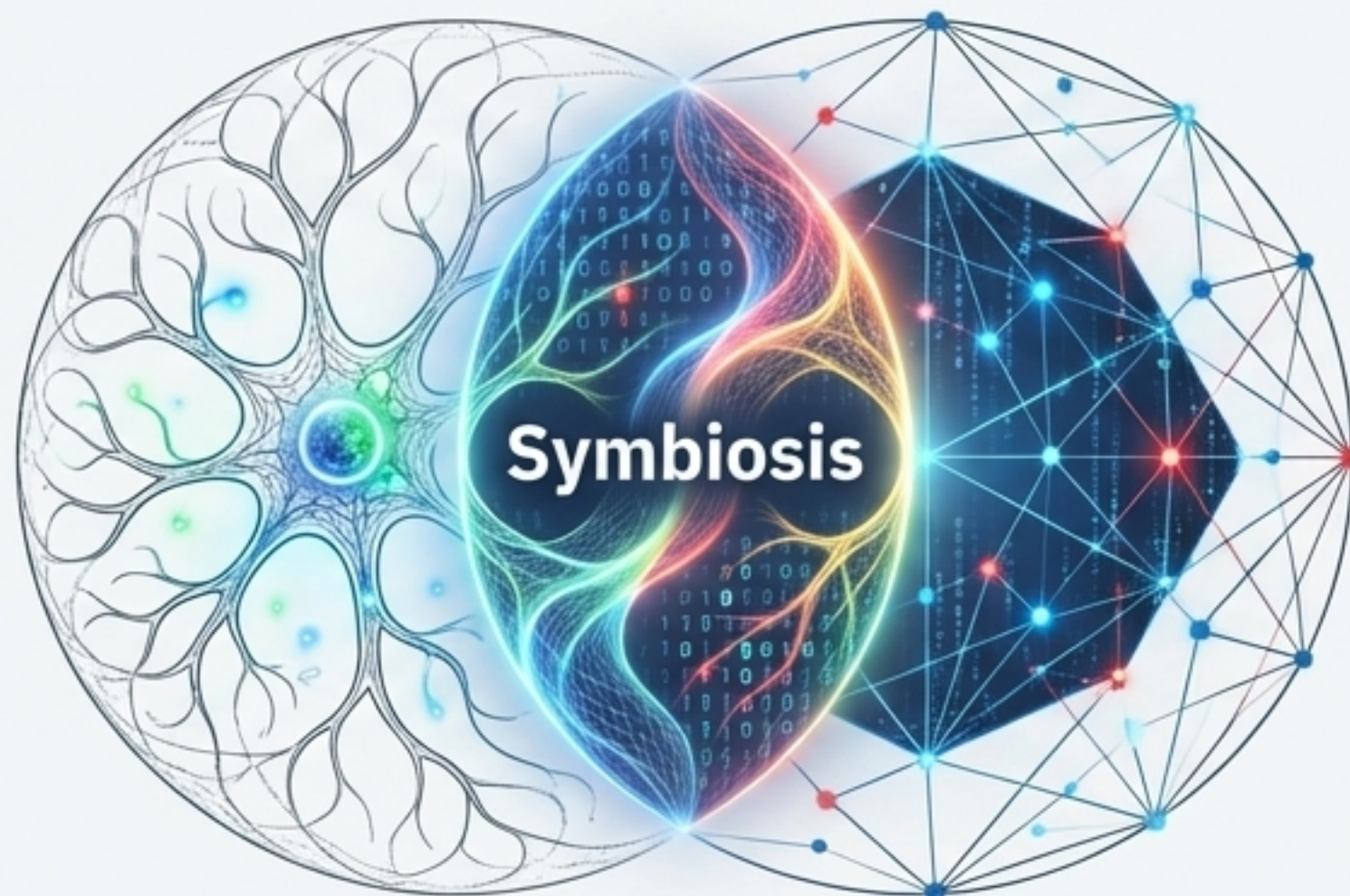


สรุปบทวิเคราะห์: เมื่อ AI มองข้าม ความสวยงาม สู่ ประสิทธิภาพ

Dimension	Human Intuition	AI Discovery
Parameter Tuning	ค่าคงที่ตายตัว (Static values)	ปรับตัวตามความผันผวนตลอดเวลา (Volatility-Adaptive)
Rule Symmetry	สมมาตร ซ้าย/ขวา บวกลบเท่ากัน (Symmetric updates)	ไม่สมมาตร ให้โบนัสพิเศษตามบริบททันที (Asymmetric boosting)
Learning Phases	เริ่มเฉลี่ยค่าตั้งแต่วินาทีแรก (Linear averaging)	ปล่อยให้ลองผิดลองถูกเต็มที่ (Hard warm-start 500 iter)
Solver Strategy	ใช้เครื่องมือเดียวแบบตายตัว (Single static solver)	ไฮบริดและปรับสัดส่วนอัตโนมัติ (Dynamic hybrid blending)

อนาคตของการออกแบบอัลกอริทึม

Human Ingenuity
(ตั้งกรอบปัญหา)



AI-driven Insights
(ค้นพบกลไก)

งานวิจัย AlphaEvolve พิสูจน์แล้วว่าเราก้าวข้ามยุค
ที่มนุษย์ต้องออกแบบทุกสมการด้วยตนเอง
LLM สามารถค้นหาเส้นทางใน Design Space
ที่กว้างใหญ่เกินจินตนาการ

อัลกอริทึมระดับ SOTA ในอนาคต
จะเกิดจากการผสมผสาน (Symbiosis)
มนุษย์รับหน้าที่ตั้งกรอบปัญหาที่เฉียบคม
และปล่อยให้ AI ค้นพบกลไกคณิตศาสตร์ที่ทรงพลังที่สุด